

4. Vezérlési szerkezetek C#-ban

Mi az a vezérlési szerkezet?

A vezérlési szerkezetek segítségével irányíthatjuk a program végrehajtásának menetét. Lehetővé teszik, hogy döntéseket hozzunk (elágazások) és műveleteket ismételjünk (ciklusok).

Elágazások (Feltételes utasítások)

if utasítás

Az `if` utasítás segítségével feltételek alapján hajtunk végre kódot.

Szintaxis:

```
if (feltétel)
{
    // Ez a kód fut le, ha a feltétel igaz
}
```

Alapvető példák:

```
int életkor = 20;

if (életkor >= 18)
{
    Console.WriteLine("Nagykorú vagy.");
}

// Egysoros if-nél elhagyható a kapcsos zárójel (de nem ajánlott!)
if (életkor >= 18)
    Console.WriteLine("Nagykorú vagy.");

// Jobb gyakorlat: mindig használj kapcsos zárójeleket
if (életkor >= 18)
{
    Console.WriteLine("Nagykorú vagy.");
}
```

Gyakorlati példák:

```
// Páros szám ellenőrzés
int szam = 10;
if (szam % 2 == 0)
{
```

```
    Console.WriteLine($"{szam} páros szám.");
}

// Jelszó hossz ellenőrzés
string jelszo = "abc12345";
if (jelszo.Length >= 8)
{
    Console.WriteLine("Jelszó elég hosszú.");
}

// Hőmérséklet ellenőrzés
int homerseklet = 35;
if (homerseklet > 30)
{
    Console.WriteLine("Meleg van!");
}

// Többsoros végrehajtás
int pontszam = 95;
if (pontszam >= 90)
{
    Console.WriteLine("Gratulálunk!");
    Console.WriteLine("Kiváló teljesítmény!");
    Console.WriteLine("Jeles osztályzat.");
}
```

if-else utasítás

Az **else** ág akkor fut le, ha a feltétel hamis.

```
int életkor = 15;

if (életkor >= 18)
{
    Console.WriteLine("Nagykorú vagy.");
}
else
{
    Console.WriteLine("Kiskorú vagy.");
}

// Gyakorlati példák

// Páros vagy páratlan
int szam = 7;
if (szam % 2 == 0)
{
    Console.WriteLine($"{szam} páros.");
}
else
{
    Console.WriteLine($"{szam} páratlan.");
}
```

```
    Console.WriteLine($"{szam} páratlan.");
}

// Pozitív vagy negatív
int ertek = -5;
if (ertek >= 0)
{
    Console.WriteLine("Pozitív vagy nulla.");
}
else
{
    Console.WriteLine("Negatív szám.");
}

// Belépés engedélyezése
bool vanJegy = false;
if (vanJegy)
{
    Console.WriteLine("Beléphet.");
}
else
{
    Console.WriteLine("Kérem, vásároljon jegyet!");
}
```

if-else if-else (többszörös elágazás)

Több feltétel vizsgálása egymás után.

```
int pontszam = 75;

if (pontszam >= 90)
{
    Console.WriteLine("Jeles (5)");
}
else if (pontszam >= 80)
{
    Console.WriteLine("Jó (4)");
}
else if (pontszam >= 70)
{
    Console.WriteLine("Közepes (3)");
}
else if (pontszam >= 60)
{
    Console.WriteLine("Elégséges (2)");
}
else
{
    Console.WriteLine("Elégtelen (1)");
}
```

```
// Hőmérséklet kategóriák
int celsius = 25;

if (celsius < 0)
{
    Console.WriteLine("Fagy van.");
}
else if (celsius < 10)
{
    Console.WriteLine("Hideg van.");
}
else if (celsius < 20)
{
    Console.WriteLine("Hűvös van.");
}
else if (celsius < 30)
{
    Console.WriteLine("Kellemes az idő.");
}
else
{
    Console.WriteLine("Meleg van.");
}

// Életszakaszok
int kor = 35;

if (kor < 2)
{
    Console.WriteLine("Csecsemő");
}
else if (kor < 12)
{
    Console.WriteLine("Gyermek");
}
else if (kor < 18)
{
    Console.WriteLine("Tizenéves");
}
else if (kor < 65)
{
    Console.WriteLine("Felnőtt");
}
else
{
    Console.WriteLine("Idős");
}
```

Beágyazott if utasítások

If utasítások egymásba ágyazása.

```
int életkor = 25;
bool vanJogositvany = true;

if (életkor >= 18)
{
    Console.WriteLine("Nagykorú vagy.");

    if (vanJogositvany)
    {
        Console.WriteLine("Vezethetsz.");
    }
    else
    {
        Console.WriteLine("Nincs jogosítványod, nem vezethetsz.");
    }
}
else
{
    Console.WriteLine("Kiskorú vagy, nem vezethetsz.");
}

// Felhasználói hozzáférés példa
string felhasznalo = "admin";
string jelszo = "1234";

if (felhasznalo == "admin")
{
    if (jelszo == "1234")
    {
        Console.WriteLine("Sikeres admin bejelentkezés!");
    }
    else
    {
        Console.WriteLine("Hibás jelszó!");
    }
}
else
{
    Console.WriteLine("Hibás felhasználónév!");
}

// Számok összehasonlítása
int a = 10;
int b = 20;
int c = 15;

if (a > b)
{
    if (a > c)
    {
        Console.WriteLine("A a legnagyobb.");
    }
    else

```

```
    {
        Console.WriteLine("C a legnagyobb.");
    }
}
else
{
    if (b > c)
    {
        Console.WriteLine("B a legnagyobb.");
    }
    else
    {
        Console.WriteLine("C a legnagyobb.");
    }
}
```

TIPP: Kerüld a túl mély beágyazást! Logikai operátorok használatával egyszerűsíthetsz:

```
// Rossz - túl mély beágyazás
if (eletkor >= 18)
{
    if (vanJogositvany)
    {
        if (vanAuto)
        {
            Console.WriteLine("Vezethetsz!");
        }
    }
}

// Jó - logikai operátorokkal
if (eletkor >= 18 && vanJogositvany && vanAuto)
{
    Console.WriteLine("Vezethetsz!");
}
```

switch utasítás

A **switch** utasítás több lehetséges érték közül választ. Olvashatóbb, mint sok if-else if.

Szintaxis:

```
switch (változó)
{
    case érték1:
        // kód
        break;
    case érték2:
```

```
        // kód
        break;
    default:
        // alapértelmezett kód
        break;
}
```

Alapvető switch példák

```
int nap = 3;

switch (nap)
{
    case 1:
        Console.WriteLine("Hétfő");
        break;
    case 2:
        Console.WriteLine("Kedd");
        break;
    case 3:
        Console.WriteLine("Szerda");
        break;
    case 4:
        Console.WriteLine("Csütörtök");
        break;
    case 5:
        Console.WriteLine("Péntek");
        break;
    case 6:
        Console.WriteLine("Szombat");
        break;
    case 7:
        Console.WriteLine("Vasárnap");
        break;
    default:
        Console.WriteLine("Érvénytelen nap!");
        break;
}

// String-el is működik
string szin = "piros";

switch (szin)
{
    case "piros":
        Console.WriteLine("A szín piros.");
        break;
    case "kék":
        Console.WriteLine("A szín kék.");
        break;
    case "zöld":
```

```
        Console.WriteLine("A szín zöld.");  
        break;  
    default:  
        Console.WriteLine("Ismeretlen szín.");  
        break;  
}
```

Több case ugyanarra az ágra

```
int honap = 12;  
  
switch (honap)  
{  
    case 12:  
    case 1:  
    case 2:  
        Console.WriteLine("Tél");  
        break;  
    case 3:  
    case 4:  
    case 5:  
        Console.WriteLine("Tavas");  
        break;  
    case 6:  
    case 7:  
    case 8:  
        Console.WriteLine("Nyár");  
        break;  
    case 9:  
    case 10:  
    case 11:  
        Console.WriteLine("Ősz");  
        break;  
    default:  
        Console.WriteLine("Érvénytelen hónap!");  
        break;  
}  
  
// Hétvége vagy hétköznap  
string nap = "szombat";  
  
switch (nap)  
{  
    case "hétfő":  
    case "kedd":  
    case "szerda":  
    case "csütörtök":  
    case "péntek":  
        Console.WriteLine("Hétköznap - Munkaidő");  
        break;  
    case "szombat":
```



```
case "vasárnap":
    Console.WriteLine("Hétféje - Pihenőnap");
    break;
default:
    Console.WriteLine("Ismeretlen nap");
    break;
}
```

Gyakorlati switch példák

```
// Számológép
char muvelet = '+';
double a = 10;
double b = 5;
double eredmeny = 0;

switch (muvelet)
{
    case '+':
        eredmeny = a + b;
        Console.WriteLine($"{a} + {b} = {eredmeny}");
        break;
    case '-':
        eredmeny = a - b;
        Console.WriteLine($"{a} - {b} = {eredmeny}");
        break;
    case '*':
        eredmeny = a * b;
        Console.WriteLine($"{a} * {b} = {eredmeny}");
        break;
    case '/':
        if (b != 0)
        {
            eredmeny = a / b;
            Console.WriteLine($"{a} / {b} = {eredmeny}");
        }
        else
        {
            Console.WriteLine("Hiba: Nullával nem lehet osztani!");
        }
        break;
    default:
        Console.WriteLine("Ismeretlen művelet!");
        break;
}

// Menü rendszer
Console.WriteLine("Válassz menüpontot:");
Console.WriteLine("1. Új fájl");
Console.WriteLine("2. Megnyitás");
Console.WriteLine("3. Mentés");
```

```
Console.WriteLine("4. Kilépés");

int választas = int.Parse(Console.ReadLine());

switch (választas)
{
    case 1:
        Console.WriteLine("Új fájl létrehozása...");
        break;
    case 2:
        Console.WriteLine("Fájl megnyitása...");
        break;
    case 3:
        Console.WriteLine("Fájl mentése...");
        break;
    case 4:
        Console.WriteLine("Kilépés...");
        break;
    default:
        Console.WriteLine("Érvénytelen választás!");
        break;
}
```

Switch kifejezés (C# 8.0+)

Rövidebb, modernebb szintaxis.

```
// Régi switch utasítás
int nap = 3;
string napNev = "";

switch (nap)
{
    case 1:
        napNev = "Hétfő";
        break;
    case 2:
        napNev = "Kedd";
        break;
    case 3:
        napNev = "Szerda";
        break;
    default:
        napNev = "Ismeretlen";
        break;
}

// Új switch kifejezés (rövidebb)
string napNev2 = nap switch
{
    1 => "Hétfő",
```

```
2 => "Kedd",
3 => "Szerda",
4 => "Csütörtök",
5 => "Péntek",
6 => "Szombat",
7 => "Vasárnap",
_ => "Ismeretlen" // _ = default
};

Console.WriteLine(napNev2);

// További példák
string evszak = hónap switch
{
    12 or 1 or 2 => "Tél",
    3 or 4 or 5 => "Tavaszi",
    6 or 7 or 8 => "Nyári",
    9 or 10 or 11 => "Ősz",
    _ => "Érvénytelen"
};

// Számológép switch kifejezéssel
double Szamol(double a, double b, char op) => op switch
{
    '+' => a + b,
    '-' => a - b,
    '*' => a * b,
    '/' => b != 0 ? a / b : 0,
    _ => 0
};

Console.WriteLine(Szamol(10, 5, '+')); // 15
```

Ciklusok (Ismétlési szerkezetek)

Ciklusok segítségével ugyanazt a kódot többször is lefuttathatjuk.

for ciklus

A **for** ciklus akkor hasznos, ha előre tudjuk, hányszor szeretnénk ismételni.

Szintaxis:

```
for (inicializálás; feltétel; léptetés)
{
    // ismétlendő kód
}
```

Alapvető példák:

```
// Számok 0-tól 4-ig
for (int i = 0; i < 5; i++)
{
    Console.WriteLine(i);
}
// Kimenet: 0, 1, 2, 3, 4

// Számok 1-től 10-ig
for (int i = 1; i <= 10; i++)
{
    Console.WriteLine(i);
}

// Páros számok 0-tól 10-ig
for (int i = 0; i <= 10; i += 2)
{
    Console.WriteLine(i);
}
// Kimenet: 0, 2, 4, 6, 8, 10

// Visszafelé számolás
for (int i = 10; i >= 1; i--)
{
    Console.WriteLine(i);
}
Console.WriteLine("Start!");

// Több lépéssel
for (int i = 0; i <= 20; i += 5)
{
    Console.WriteLine(i);
}
// Kimenet: 0, 5, 10, 15, 20
```

Gyakorlati példák:

```
// Szorzótábla
int szam = 7;
Console.WriteLine($"{szam}-es szorzótábla:");
for (int i = 1; i <= 10; i++)
{
    Console.WriteLine($"{szam} x {i} = {szam * i}");
}

// Összeg számítás
int osszeg = 0;
for (int i = 1; i <= 100; i++)
{
    osszeg += i;
}
```

```
Console.WriteLine($"1-től 100-ig összeg: {osszeg}"); // 5050

// Faktoriális számítás
int n = 5;
int faktorialis = 1;
for (int i = 1; i <= n; i++)
{
    faktorialis *= i;
}
Console.WriteLine($"{n}! = {faktorialis}"); // 120

// Csillagok kiírása
for (int i = 1; i <= 5; i++)
{
    for (int j = 1; j <= i; j++)
    {
        Console.Write("*");
    }
    Console.WriteLine();
}
/* Kimenet:
*
**
***
****
*****
*/

// Tömb bejárása
int[] szamok = { 10, 20, 30, 40, 50 };
for (int i = 0; i < szamok.Length; i++)
{
    Console.WriteLine($"Index {i}: {szamok[i]}");
}
```

Beágyazott for ciklusok

```
// Szorzótábla teljes
for (int i = 1; i <= 10; i++)
{
    for (int j = 1; j <= 10; j++)
    {
        Console.Write($"{i * j,4}"); // 4 karakter széles
    }
    Console.WriteLine();
}

// Négyzet rajzolása
int meret = 5;
for (int i = 0; i < meret; i++)
{
```

```
    for (int j = 0; j < meret; j++)
    {
        Console.Write("* ");
    }
    Console.WriteLine();
}
/* Kimenet:
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
*/

// Háromszög
for (int i = 1; i <= 5; i++)
{
    for (int j = 1; j <= i; j++)
    {
        Console.Write(j + " ");
    }
    Console.WriteLine();
}
/* Kimenet:
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
*/
```

while ciklus

A **while** ciklus addig ismétlődik, amíg a feltétel igaz. Akkor használjuk, ha nem tudjuk előre, hányszor kell ismételni.

Szintaxis:

```
while (feltétel)
{
    // ismétlendő kód
}
```

Alapvető példák:

```
// Számolás 0-tól 4-ig
int i = 0;
while (i < 5)
```

```
{
    Console.WriteLine(i);
    i++;
}
// Kimenet: 0, 1, 2, 3, 4

// Visszafelé számolás
int szamlalo = 10;
while (szamlalo > 0)
{
    Console.WriteLine(szamlalo);
    szamlalo--;
}
Console.WriteLine("Start!");

// Feltétel alapú ismétlés
int szam = 1;
while (szam <= 100)
{
    Console.WriteLine(szam);
    szam *= 2; // Duplázás
}
// Kimenet: 1, 2, 4, 8, 16, 32, 64
```

Gyakorlati példák:

```
// Felhasználói input bekérése, amíg nem valid
string jelszo = "";
while (jelszo.Length < 8)
{
    Console.Write("Add meg a jelszót (min. 8 karakter): ");
    jelszo = Console.ReadLine();

    if (jelszo.Length < 8)
    {
        Console.WriteLine("A jelszó túl rövid!");
    }
}
Console.WriteLine("Jelszó elfogadva!");

// Számok összege, amíg el nem érjük a limitet
int osszeg = 0;
int szamlalo = 1;
int limit = 100;

while (osszeg < limit)
{
    osszeg += szamlalo;
    Console.WriteLine($"Szám: {szamlalo}, Összeg: {osszeg}");
    szamlalo++;
}
```

```
// Véletlenszám generálás, amíg nem találunk egy meghatározottat
Random rnd = new Random();
int talalat = 0;
int probalkozas = 0;

while (talalat != 6)
{
    talalat = rnd.Next(1, 7); // 1-6 között
    probalkozas++;
    Console.WriteLine($"Próbálkozás {probalkozas}: {talalat}");
}
Console.WriteLine($"Hatos dobva {probalkozas} próbálkozás után!");

// Menü rendszer
bool futtat = true;
while (futtat)
{
    Console.WriteLine("\n=== MENÜ ===");
    Console.WriteLine("1. Opció 1");
    Console.WriteLine("2. Opció 2");
    Console.WriteLine("3. Kilépés");
    Console.Write("Választás: ");

    int valasztas = int.Parse(Console.ReadLine());

    switch (valasztas)
    {
        case 1:
            Console.WriteLine("Opció 1 kiválasztva");
            break;
        case 2:
            Console.WriteLine("Opció 2 kiválasztva");
            break;
        case 3:
            futtat = false;
            Console.WriteLine("Kilépés...");
            break;
        default:
            Console.WriteLine("Érvénytelen választás!");
            break;
    }
}
```

FIGYELEM: Végtelen ciklus!

```
// ROSSZ - végtelen ciklus, mert i sosem változik
int i = 0;
while (i < 5)
{
    Console.WriteLine(i);
    // HIÁNYZIK: i++;
}
```



```
// ROSSZ - a feltétel mindig igaz
while (true)
{
    Console.WriteLine("Végtelen...");
}

// JÓ - van kilépési feltétel
while (true)
{
    Console.Write("Írd be 'kilép' szót: ");
    string input = Console.ReadLine();

    if (input == "kilép")
    {
        break; // Kilépés a ciklusból
    }
}
```

do-while ciklus

A **do-while** ciklus legalább egyszer lefut, utána ellenőrzi a feltételt.

Szintaxis:

```
do
{
    // ismétlendő kód
} while (feltétel);
```

Különbség a while-tól:

```
// while - nem fut le, ha a feltétel eleve hamis
int i = 10;
while (i < 5)
{
    Console.WriteLine("Ez nem fut le");
    i++;
}

// do-while - legalább egyszer lefut
int j = 10;
do
{
    Console.WriteLine("Ez egyszer lefut"); // Lefut!
    j++;
} while (j < 5);
```

Gyakorlati példák:

```
// Menü, ami legalább egyszer megjelenik
int választas;
do
{
    Console.WriteLine("\n=== MENÜ ===");
    Console.WriteLine("1. Új játék");
    Console.WriteLine("2. Betöltés");
    Console.WriteLine("3. Beállítások");
    Console.WriteLine("4. Kilépés");
    Console.Write("Választás: ");
    választas = int.Parse(Console.ReadLine());

    switch (választas)
    {
        case 1:
            Console.WriteLine("Új játék indítása...");
            break;
        case 2:
            Console.WriteLine("Játék betöltése...");
            break;
        case 3:
            Console.WriteLine("Beállítások...");
            break;
        case 4:
            Console.WriteLine("Viszlát!");
            break;
        default:
            Console.WriteLine("Érvénytelen választás!");
            break;
    }
} while (választas != 4);

// Szám bekérése, amíg nem valid
int szam;
do
{
    Console.Write("Adj meg egy pozitív számot: ");
    szam = int.Parse(Console.ReadLine());

    if (szam <= 0)
    {
        Console.WriteLine("A számnak pozitívnak kell lennie!");
    }
} while (szam <= 0);

Console.WriteLine($"Köszönöm, a szám: {szam}");

// Jelszó ellenőrzés próbálkozásokkal
string helyesJelszo = "titok123";
string bemenet;
int probalkozasok = 0;
```

```
int maxProbalkozas = 3;

do
{
    probalkozasok++;
    Console.WriteLine($"Add meg a jelszót ({probalkozasok}/{maxProbalkozas}): ");
    bemenet = Console.ReadLine();

    if (bemenet != helyesJelszo)
    {
        Console.WriteLine("Hibás jelszó!");
    }
} while (bemenet != helyesJelszo && probalkozasok < maxProbalkozas);

if (bemenet == helyesJelszo)
{
    Console.WriteLine("Sikeres bejelentkezés!");
}
else
{
    Console.WriteLine("Túl sok sikertelen próbálkozás!");
}
```

foreach ciklus

A **foreach** ciklus gyűjtemények (tömbök, listák) elemeinek bejárására szolgál.

Szintaxis:

```
foreach (típus elem in gyűjtemény)
{
    // művelet az elemmel
}
```

Alapvető példák:

```
// Tömb bejárása
int[] szamok = { 1, 2, 3, 4, 5 };

foreach (int szam in szamok)
{
    Console.WriteLine(szam);
}

// String tömb
string[] nevek = { "Anna", "Béla", "Cecil", "Dóra" };

foreach (string nev in nevek)
```

```
{
    Console.WriteLine($"Szia, {nev}!");
}

// String karaktereinek bejárása
string szoveg = "Hello";

foreach (char betu in szoveg)
{
    Console.WriteLine(betu);
}

// Kimenet: H, e, l, l, o
```

foreach vs for különbség:

```
int[] szamok = { 10, 20, 30, 40, 50 };

// for - van indexünk
for (int i = 0; i < szamok.Length; i++)
{
    Console.WriteLine($"Index {i}: {szamok[i]}");
    szamok[i] = szamok[i] * 2; // Módosíthatjuk
}

// foreach - nincs index, csak értékünk
foreach (int szam in szamok)
{
    Console.WriteLine(szam);
    // szam = szam * 2; // NEM működik! Csak olvasható
}

// Ha szükséges az index foreach-nél
int index = 0;
foreach (string nev in nevek)
{
    Console.WriteLine($"{index}. {nev}");
    index++;
}
```

Gyakorlati példák:

```
// Összeg számítás
int[] szamok = { 5, 10, 15, 20, 25 };
int osszeg = 0;

foreach (int szam in szamok)
{
    osszeg += szam;
}
```

```
Console.WriteLine($"Összeg: {osszeg}"); // 75

// Maximum keresés
int[] ertekek = { 34, 12, 78, 23, 90, 45 };
int max = ertekek[0];

foreach (int ertek in ertekek)
{
    if (ertek > max)
    {
        max = ertek;
    }
}
Console.WriteLine($"Maximum: {max}"); // 90

// Páros számok kiírása
int[] lista = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };

Console.WriteLine("Páros számok:");
foreach (int szam in lista)
{
    if (szam % 2 == 0)
    {
        Console.WriteLine(szam);
    }
}

// Szavak hosszának vizsgálata
string[] szavak = { "alma", "körte", "barack", "szilva" };

foreach (string szo in szavak)
{
    Console.WriteLine($"{szo} - {szo.Length} karakter");
}
```

break és continue utasítások

Ciklusok vezérlésére szolgáló speciális utasítások.

break utasítás

A **break** kilép a ciklusból (megszakítja azt).

```
// Szám keresése tömbben
int[] szamok = { 10, 20, 30, 40, 50 };
int keresett = 30;
bool megtalalta = false;

for (int i = 0; i < szamok.Length; i++)
{
```

```
        if (szamok[i] == keresett)
        {
            Console.WriteLine($"Megtaláltam a {keresett}-at az {i}. indexen!");
            megtalalta = true;
            break; // Kilép, nem keres tovább
        }
    }

    if (!megtalalta)
    {
        Console.WriteLine("Nem találtam.");
    }

    // Végtelen ciklus break-kel
    int szamlalo = 0;
    while (true)
    {
        Console.WriteLine(szamlalo);
        szamlalo++;

        if (szamlalo >= 5)
        {
            break; // Kilép 5-nél
        }
    }

    // Felhasználói input
    while (true)
    {
        Console.Write("Írj be egy számot (0 = kilépés): ");
        int szam = int.Parse(Console.ReadLine());

        if (szam == 0)
        {
            Console.WriteLine("Kilépés...");
            break;
        }

        Console.WriteLine($"Beírtad: {szam}");
    }

    // Jelszó ellenőrzés max próbálkozással
    string helyes = "abc123";
    int maxProba = 3;

    for (int i = 1; i <= maxProba; i++)
    {
        Console.Write($"Jelszó ({i}/{maxProba}): ");
        string input = Console.ReadLine();

        if (input == helyes)
        {
            Console.WriteLine("Helyes jelszó!");
            break;
        }
    }
}
```

```
}

Console.WriteLine("Hibás jelszó!");
}
```

continue utasítás

A `continue` átugorja az aktuális iterációt, és a következővel folytatja.

```
// Páros számok kihagyása
for (int i = 1; i <= 10; i++)
{
    if (i % 2 == 0)
    {
        continue; // Páros számokat kihagyja
    }
    Console.WriteLine(i); // Csak páratlanokat írja ki
}
// Kimenet: 1, 3, 5, 7, 9

// Negatív számok kihagyása
int[] szamok = { -5, 10, -3, 7, 0, 15, -8 };

foreach (int szam in szamok)
{
    if (szam < 0)
    {
        continue; // Negatívakat átugorja
    }
    Console.WriteLine(szam);
}
// Kimenet: 10, 7, 0, 15

// Üres stringek átugrása
string[] nevek = { "Anna", "", "Béla", "", "Cecil" };

foreach (string nev in nevek)
{
    if (nev == "")
    {
        continue;
    }
    Console.WriteLine($"Üdv, {nev}!");
}
// Kimenet: Üdv, Anna! Üdv, Béla! Üdv, Cecil!

// Oszthatóság ellenőrzés
for (int i = 1; i <= 20; i++)
{
    if (i % 3 != 0) // Ha nem osztható 3-mal
    {

```

```
        continue; // Ugord át
    }
    Console.WriteLine($"{i} osztható 3-mal");
}
// Kimenet: 3, 6, 9, 12, 15, 18
```

break vs continue összehasonlítás

```
// break példa - megáll az első 5-nél
Console.WriteLine("break példa:");
for (int i = 1; i <= 10; i++)
{
    if (i == 5)
    {
        break; // Kilép a ciklusból
    }
    Console.WriteLine(i);
}
// Kimenet: 1, 2, 3, 4

Console.WriteLine("\ncontinue példa:");
// continue példa - kihagyja az 5-öt
for (int i = 1; i <= 10; i++)
{
    if (i == 5)
    {
        continue; // Átugorja az 5-öt
    }
    Console.WriteLine(i);
}
// Kimenet: 1, 2, 3, 4, 6, 7, 8, 9, 10
```

Összetett gyakorlati példák

1. Prímszám ellenőrző

```
Console.Write("Add meg a számot: ");
int szam = int.Parse(Console.ReadLine());

bool prim = true;

if (szam < 2)
{
    prim = false;
}
else
{
    for (int i = 2; i <= Math.Sqrt(szam); i++)
```



```
{
    if (szam % i == 0)
    {
        prim = false;
        break;
    }
}

if (prim)
{
    Console.WriteLine($"{szam} prímszám.");
}
else
{
    Console.WriteLine($"{szam} nem prímszám.");
}
```

2. Számkitalálós játék

```
Random rnd = new Random();
int titkosSzam = rnd.Next(1, 101); // 1-100 között
int tippek = 0;
int maxTipp = 7;
bool talalt = false;

Console.WriteLine("Gondoltam egy számra 1 és 100 között!");
Console.WriteLine($"Neked {maxTipp} tipped van.");

while (tippek < maxTipp && !talalt)
{
    tippek++;
    Console.Write($"\\n{tippek}. tipp: ");
    int tipp = int.Parse(Console.ReadLine());

    if (tipp == titkosSzam)
    {
        Console.WriteLine($"Gratulálok! Eltaláltad {tippek} próbálkozásból!");
        talalt = true;
    }
    else if (tipp < titkosSzam)
    {
        Console.WriteLine("Nagyobb!");
    }
    else
    {
        Console.WriteLine("Kisebb!");
    }
}

if (!talalt)
```

```
{  
    Console.WriteLine($"Sajnálom, elfogytak a tippjeid! A szám {titkosSzam}  
volt.");  
}
```

3. Számlálás karakterláncban

```
Console.Write("Írj be egy szöveget: ");  
string szoveg = Console.ReadLine();  
  
int maganhangzok = 0;  
int massalhangzok = 0;  
int szamjegyek = 0;  
int szokozok = 0;  
  
foreach (char c in szoveg.ToLower())  
{  
    if ("aeiouáéíóőúü".Contains(c))  
    {  
        maganhangzok++;  
    }  
    else if (char.IsLetter(c))  
    {  
        massalhangzok++;  
    }  
    else if (char.IsDigit(c))  
    {  
        szamjegyek++;  
    }  
    else if (c == ' ')  
    {  
        szokozok++;  
    }  
}  
  
Console.WriteLine($"Magánhangzók: {maganhangzok}");  
Console.WriteLine($"Mássalhangzók: {massalhangzok}");  
Console.WriteLine($"Számjegyek: {szamjegyek}");  
Console.WriteLine($"Szóközök: {szokozok}");
```

4. FizzBuzz játék

```
// Klasszikus programozói feladat  
for (int i = 1; i <= 100; i++)  
{  
    if (i % 15 == 0) // Osztható 3-mal ÉS 5-tel  
    {  
        Console.WriteLine("FizzBuzz");  
    }  
}
```

```
    else if (i % 3 == 0)
    {
        Console.WriteLine("Fizz");
    }
    else if (i % 5 == 0)
    {
        Console.WriteLine("Buzz");
    }
    else
    {
        Console.WriteLine(i);
    }
}
```

Összefoglalás

Elágazások

- **if**: Feltételes végrehajtás
- **if-else**: Kétirányú elágazás
- **if-else if-else**: Többszörös elágazás
- **switch**: Többirányú elágazás fix értékekre

Ciklusok

- **for**: Ismert számú ismétlés, indexszel
- **while**: Feltételfüggő ismétlés, előtesztelés
- **do-while**: Legalább egyszer lefut, utótesztelés
- **foreach**: Gyűjtemények bejárása

Vezérlő utasítások

- **break**: Kilép a ciklusból
- **continue**: Átugorja az aktuális iterációt

Fontos:

- Használj kapcsos zárójeleket mindig
- Kerüld a végtelen ciklusokat
- A foreach csak olvasásra, a for módosításra is jó
- break és continue csak ciklusokban használható